Computer Vision - 2026

Week #12. Vision - Language - Action Models

Lectures by Alexei Kornaev [1,2]
Practical sessions by Kirill Yakovlev [1,3]

[1]The Center for Top-Level Educational Programs in AI, Innopolis University (IU), Innopolis
[2]RC for AI, National RC for Oncology n.a. N.N.Blokhin, Moscow
[3]Ak Bars Bank, Kazan

Agenda

1. Modern VLA Architecture
2. Training Modern VLA Systems
3. Modern VLA Post-Training with Reinforcement Learning

From Perception to Action

Computer vision traditionally focused on prediction:

$$\text{Image} \rightarrow \text{Label}$$

Modern embodied systems require control:

$$\text{Observation} \rightarrow \text{Decision} \rightarrow \text{Action} \rightarrow \text{Feedback}$$

The system must continuously interact with the environment.

- perception estimates the current state
- action modifies the environment
- feedback determines the next decision

This interaction loop defines intelligent behavior in physical systems.

Recognition Is Not Control

Recognizing an object does not imply the ability to manipulate it.

Example:

- a model detects a cup
- the robot must decide:
  - where to grasp
  - how much force to apply
  - how to move safely

The central challenge becomes:

What action should be executed next?

This shift transforms vision systems into control systems.

What Is a Vision-Language-Action Model

A Vision-Language-Action (VLA) model extends perception with decision-making.

Input:

$$s_t = (\text{visual observation}, \ \text{robot state}, \ \text{task instruction})$$

Output:

$$a_t = \text{control command}$$

The model predicts an action that changes the physical world.

- vision provides information
- language defines the goal
- action executes the plan

Closed-Loop Control

Unlike image classification, robotic systems operate in a loop.

$$s_t \rightarrow a_t \rightarrow s_{t+1} \rightarrow a_{t+1} \rightarrow \cdots$$

Each action changes the environment and produces new observations.

- the system observes
- the system acts
- the environment responds
- the cycle repeats

Reliable behavior therefore requires continuous adaptation.

The Core Question of Embodied Intelligence

Modern AI systems must operate in uncertain and dynamic environments.

The fundamental decision problem is:

$$a^* = \arg\max_a \; P(\text{success} \mid s, a)$$

The system selects the action most likely to achieve the goal.

- perception estimates the situation
- planning evaluates alternatives
- control executes the selected action
- feedback validates the result

Why Vision-Language-Action Models Matter

VLA systems represent the next stage of AI development.
They enable machines to:

Capabilities

- manipulate objects
- follow natural language instructions
- adapt to changing environments
- operate autonomously

Applications

- household robots
- warehouse automation
- medical assistance systems
- industrial inspection

The focus shifts from prediction to reliable behavior.

Learning Outcomes

After completing this lecture, students should be able to:

- **Explain** what a **Vision-Language-Action (VLA)** model is and how it differs from traditional perception systems
- **Describe** the modern VLA architecture pipeline:

$$\text{observation} \longrightarrow \text{representation} \longrightarrow \text{policy} \longrightarrow \text{action}$$

- **Identify** the main stages of training a modern robot policy:
  - foundation model pretraining
  - supervised imitation learning
  - reinforcement learning post-training
- **Interpret** the role of large-scale robot datasets (e.g., LIBERO, RoboTwin) in improving generalization
- **Analyze** how reinforcement learning can improve performance after supervised training
- **Evaluate** the difference between imitation and exploration in robot learning
- **Recognize** the emerging pattern of capability growth through interaction rather than manual supervision

Operational skill

Students should be able to read a modern VLA paper and reconstruct its training pipeline and evaluation logic.

Section 1. Modern VLA Architecture

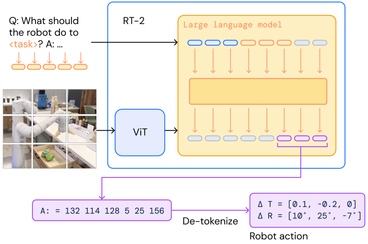
Modern VLA Architecture: The Main Question

A VLA maps multimodal context to executable control:

$$(\text{image}, \ \text{robot state}, \ \text{instruction}) \longrightarrow \text{action}$$

Why this is harder than perception-only models

- integrates visual scene understanding with language goals
- estimates physical state and predicts robot-safe commands
- must adapt online from feedback after each action step



Core architecture choices

- **Encoder:** CNN / ViT / multimodal perception backbone
- **Fusion:** concatenation, cross-attention, autoregressive mixing
- **Action head:** continuous control, tokenized actions, or diffusion chunks
- **Rollout:** open-loop chunking vs. closed-loop replanning

Canonical references: RT-1 Brohan et al. [2022], RT-2 Brohan et al. [2023], Open X-Embodiment Collaboration [2023], llaboration et al. [2024], OpenVLA Kim et al. [2024a], Diffusion Policy Chi et al. [2023], π_0 Black et al. [2024].

Canonical VLA Pipeline

Operational view

A modern VLA follows a four-stage mapping:

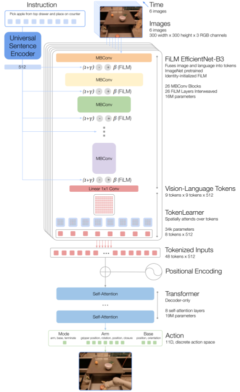
Perception Encoder $\longrightarrow$ Multimodal Fusion $\longrightarrow$ Policy Backbone

Policy Backbone $\longrightarrow$ Action Decoder

Inputs and outputs

- **Inputs:** RGB image(s), optional depth/point cloud, proprioception, language goal, history
- **Outputs:** end-effector pose delta, joint action vector, gripper command, action chunk
- Action format may be continuous, tokenized, or denoised (diffusion-style)

This abstraction covers RT-1, RT-2, Octo, and OpenVLA Brohan et al. [2022, 2023]; Collaboration et al. [2024]; Kim et al. [2024a].



Architecture Taxonomy: Three Main Families

Family	Main Idea	Strength	Representative Models
Transformer BC policy	Autoregressive or sequence policy maps observations to actions	Simple scaling, unified token pipeline	RT-1 Brohan et al. [2022], RT-2 Brohan et al. [2023], OpenVLA Kim et al. [2024a]
Diffusion action policy	Predict action chunk by iterative denoising	Smooth multimodal trajectories, robust low-level control	Diffusion Policy Chi et al. [2023], RAD / RDT-style descendants
Generalist multi-embodiment policy	Train one policy on many robots, tasks, embodiments	Transfer, broader coverage, better data reuse	Octo Collaboration et al. [2024], Open X-Embodiment Collaboration [2023], π_0 Black et al. [2024]

Practical interpretation

The field is converging to a common recipe:

$$\text{foundation-style perception} + \text{language conditioning} + \text{robot policy head}$$

while differing mainly in **how actions are represented and decoded**.

What Does the Model Actually Predict?

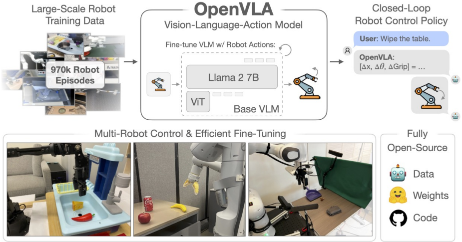
Representation	Model output	Strengths	Trade-offs
Continuous control	$a_t \in \mathbb{R}^d$ (pose/joint/gripper)	direct execution; simple head	harder multimodal prediction; less stable for diverse trajectories
Action tokens	$a_t \to z_t \in \mathcal{V}_{\text{act}}$	LLM-compatible training; autoregressive decoding	quantization error; tokenizer design matters
Diffusion chunks	$(a_t, \ldots, a_{t+H-1}) \sim p_\theta(\cdot \mid s_t)$	smooth trajectories; strong low-level control; multimodality	slower decoding; more complex inference

Rule of thumb

Token policies are best for **foundation-model scaling**; diffusion policies for **fine motor control**; continuous heads for **small, efficient systems**.

This taxonomy appears repeatedly across recent VLA systems Brohan et al. [2022, 2023]; Chi et al. [2023]; Kim et al. [2024a]; Black et al. [2024].

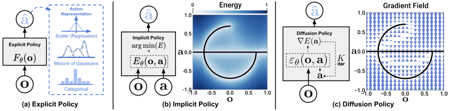
Two Practical Blueprints



A. Token-based VLA

$(I_t, q_t, l) \rightarrow$ Transformer $\rightarrow$ action tokens $\rightarrow$ dequantize

Typical for RT-style and OpenVLA-style systems.



B. Diffusion-based VLA

$(I_t, q_t, l) \rightarrow$ conditioned denoiser $\rightarrow$ action chunk

Typical for Diffusion Policy and related control models.

difference

Token policies look more like language models. Diffusion policies look more like generative control models. Both implement the

Architecture: Putting the Pieces Together

Perception side

- RGB image encoder: CNN or ViT
- optional depth / point cloud encoder
- optional object-centric features
- temporal stacking or history tokens

Language side

- instruction tokens
- goal description
- sometimes dialogue or plan tokens
- may reuse VLM / LLM tokenizer and backbone

Fusion

- concatenate image + text + proprioception tokens
- cross-attention between modalities
- usal decoding over mixed token stream

Policy side

- transformer backbone
- recurrent memory or history window
- predicts next action or action chunk

Execution side

- action decoder
- safety or clipping layer
- simulator / robot controller
- next state returned to the model

Design tension

A strong VLA must balance:

- general semantic knowledge
- spatial precision
- low-latency control
- robustness under distribution shift

Inference Algorithm for a Closed-Loop VLA

Algorithm 1: Generic inference loop for a modern VLA policy

Input: Instruction I, initial observation o_0, robot state q_0, policy π_θ
Output: Executed trajectory τ

Initialize environment and set task goal
$\tau \leftarrow \emptyset$
for $t = 0, 1, \ldots, T-1$ **do**
  Encode visual observation $f_t^{vis} \leftarrow E_{vis}(o_t)$
  Encode robot state $f_t^{prop} \leftarrow E_{prop}(q_t)$
  Encode language instruction $f^{lang} \leftarrow E_{lang}(I)$
  Fuse multimodal context:

$$h_t \leftarrow \mathrm{Fuse}(f_t^{vis}, f_t^{prop}, f^{lang}, \tau_{<t})$$

  Predict action representation:

$$z_t \leftarrow \pi_\theta(h_t)$$

  Decode executable action:

$$a_t \leftarrow \mathrm{Decode}(z_t)$$

  Execute a_t in simulator / robot controller
  Observe next state (o_{t+1}, q_{t+1})
  Append (o_t, q_t, a_t) to τ
  if **task success or termination** **then**
    break
  end
end

Representative Architectures

Model	Backbone	Action Head	Main Message
RT-1 Brohan et al. [2022]	Token-based robot transformer	discretized action tokens	Scaling supervised robot policies works
RT-2 Brohan et al. [2023]	VLM adapted to robotic control	text/action token style decoding	Web knowledge can help action generation
Octo Collaboration et al. [2024]	Generalist transformer policy	action chunk prediction	Multi-embodiment data improves transfer
OpenVLA Kim et al. [2024a]	Open vision-language-action backbone	7-DoF action prediction	Open-source foundation policy for manipulation
Diffusion Policy Chi et al. [2023]	Conditioned diffusion model	denoised action trajectory	Generative control is strong for manipulation
π_0 Black et al. [2024]	Large-scale generalist robot policy	high-capacity action model	Scale and breadth are becoming central

Teaching note

Students do not need every detail here. They should learn to identify: **backbone, fusion strategy, action representation, rollout loop**.

Paper Reading 1: RT-1 and RT-2

RT-1 and **RT-2** established the transformer line of VLA systems.

What to notice

- RT-1 treats robot control as sequence modeling
- RT-2 pushes further: a vision-language model is adapted to output robot actions
- the key conceptual shift is that semantic knowledge from large pretraining may help embodied control

Reading focus

While reading, identify:

- how actions are tokenized
- what enters the transformer context
- whether the model predicts one action or a chunk
- what kind of scaling claim the paper makes

Suggested reading: RT-1: Robotics Transformer for Real-World Control at Scale and RT-2: Vision-Language-Action Models Web Knowledge to Robotic Control.

Paper Reading 2: OpenVLA and Octo

OpenVLA and **Octo** represent the open, generalist direction of the field.

OpenVLA

What to notice:

- open implementation matters
- VLA becomes easier to study in labs
- action generation is integrated with vision-language backbone design

Octo

What to notice:

- cross-embodiment training
- generalist policy viewpoint
- data standardization is part of the architecture story

Main lesson

Architecture is no longer only about layers. It also includes the **training interface**, **data format**, and **robot embodiment abstraction**.

Suggested reading: OpenVLA: An Open Vision-Language-Action Model and Octo: An Open-Source Generalist Robot Policy.

Paper Reading 3: Diffusion Policy

Diffusion Policy is the cleanest example of a generative control architecture.

Why it matters

It shows that robot action prediction is not only a classification or regression problem. It can also be treated as **trajectory generation**.

What to notice while reading

- actions are predicted as a short future sequence
- denoising models naturally represent multimodal solutions
- the architecture is often easier to connect to manipulation than large VLM-style token pipelines

Course connection

This is the direct bridge from your earlier lecture on diffusion / generative modeling to embodied control.

Suggested reading: Diffusion Policy: Visuomotor Policy Learning via Action Diffusion.

Main Takeaways of the Architecture Section

Three messages

1. A VLA architecture is a **closed-loop control system**, not only a multimodal predictor.
2. The main architectural choice is usually the **action representation**: continuous, tokenized, or diffusion-based.
3. Recent models differ less in high-level pipeline structure than in **scaling strategy, data regime, and action decoding**.

Compact summary

Perception + Language + Robot State $\rightarrow$ Policy Backbone $\rightarrow$ Executable Actionthen repeat in a loop

Once the architecture is defined, the next question is:

How do we train such systems well?

This leads naturally to imitation learning, action chunking, scaling data, and RL refinement.

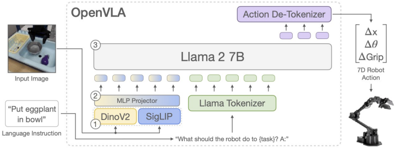
Section 2. Training Modern VLA Systems

From Models to Policies

Modern VLA training is staged rather than one-shot.

Modern training stack

1. foundation model pretraining
2. multi-robot data-mixture training
3. task-specific fine-tuning
4. deployment-time adaptation



Representative systems

OpenVLA, $\pi_{0.5}$, SmolVLA, PixelVLA

The Modern VLA Training Objective

A VLA model learns a policy from demonstrations.

$$\pi(a \mid s, g)$$

where:

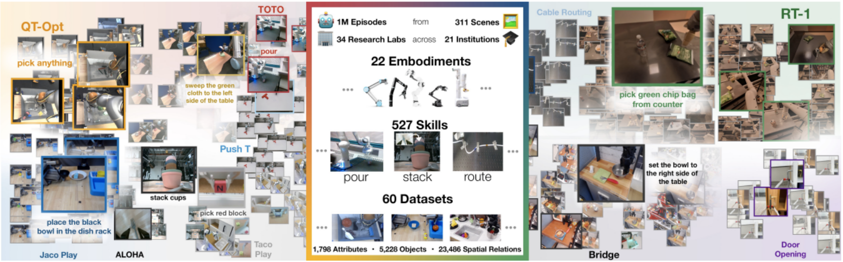
- s — observation
- g — instruction
- a — action trajectory

Training data therefore consists of:

(image, language, trajectory)

Large-scale robot datasets pair observations and instructions with trajectories collected from teleoperation or simulation.

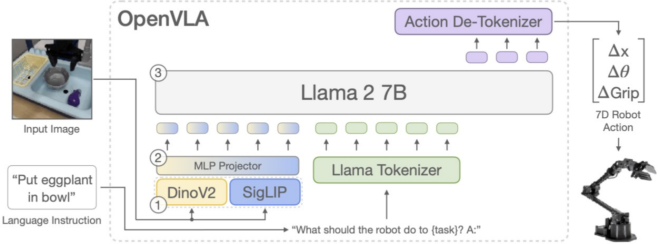
Modern VLA Datasets



Core datasets

- Open X-Embodiment Collaboration et al. [2025]
- BridgeData V2
- LIBERO simulation benchmark
- LeRobot community dataset

OpenVLA Training Pipeline



Data scale

- 970k robot demonstrations
- multiple robot embodiments
- multi-task manipulation

A uses a pretrained language model and visual encoder and is fine-tuned on diverse robot demonstrations. Kim et al. [2024b]

The $\pi_{0.5}$ Training Strategy

Core idea

Co-training on heterogeneous data sources.

Data mixture

- robot demonstrations
- language data
- perception tasks
- semantic prediction

$\pi_{0.5}$ combines these sources to improve generalization in new environments. Intelligence [2025]

Fig. 1: The $\pi_{0.5}$ model transfers knowledge from a heterogeneous range of data sources, including other robots, high-level subtask prediction, verbal instructions, and data from the web, in order to enable broad generalization across environments and objects. $\pi_{0.5}$ can control a mobile manipulator to clean kitchens and bedrooms in new homes that were not present in the training data, performing complex multi-stage behaviors with durations of 10 to 15 minutes.

Training Algorithm: Modern VLA

Algorithm 2: Modern VLA training loop

Input: Dataset of demonstrations $\mathcal{D}$
Output: Trained VLA policy π_θ

Load pretrained model parameters
Initialize policy head and optimizer
for each training step $t = 1, \ldots, T$ **do**
    Sample mini-batch from $\mathcal{D}$
    Predict action sequence $\hat{a}_{1:H}$
    Compute objective $\mathcal{L}(\hat{a}_{1:H}, a_{1:H})$
    Update parameters $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
end
Evaluate policy in simulation
Deploy the validated policy to robot controller

Paper Reading: Modern VLA Training

Suggested reading

- OpenVLA (2024)
- $\pi_{0.5}$ (2025)
- PixelVLA (2025)

What to notice

- data scale
- training stages
- generalization strategy
- dataset mixture

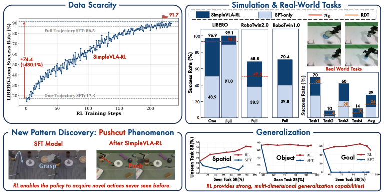
Section 3. Modern VLA Post-Training with Reinforcement Learning

Why Post-Training for VLA?

Modern VLAs already have strong perception and policy backbones, but scaling pure supervised fine-tuning (SFT) is difficult.

Two main bottlenecks

- **Data scarcity:** large numbers of human teleoperation trajectories are expensive
- **Poor generalization:** SFT policies degrade under unseen tasks, scenes, and objects

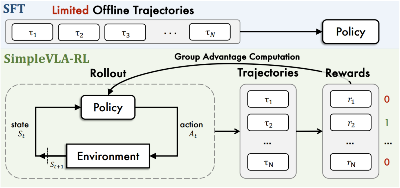


Modern idea

Do not redesign the whole architecture. Take a strong pretrained VLA and **improve it by online RL** in simulation.

The motivation and the high-level summary follow Anonymous and Others [2025]. The paper explicitly frames VLA scaling in terms of data scarcity and generalization, then asks whether RL can improve long-horizon action planning after SFT.

From SFT to RL Post-Training



Training shift

The training recipe becomes:

$$\text{Pretraining} \longrightarrow \text{SFT} \longrightarrow \text{Online RL}$$

Rather than collecting ever more demonstrations, the policy improves through interaction with the simulator.

Key change

The policy is no longer only imitating expert actions. It explores, receives rewards, and updates its action distribution.

This is the central transition proposed in Anonymous and Others [2025]. Figure 2 of the paper is a very good source for this slide.

RL Formulation for VLA

State

$$s_t = (o_t^{vis},\ o_t^{prop},\ l_{\text{task}})$$

- visual observation
- proprioception
- language instruction

Action

$$a_t \in \mathbb{R}^d$$

or tokenized / chunked action output.

- end-effector delta
- joint target
- gripper signal
- action chunk

Environment

The robot acts in a simulator:

$$s_{t+1} = \mathrm{Env}(s_t, a_t)$$

Each action changes the observation.

Rollout

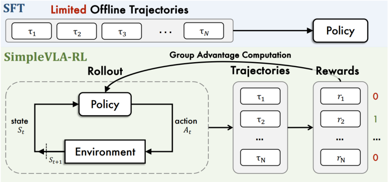
The policy repeatedly:

$$s_t \to a_t \to s_{t+1}$$

This is closed-loop control, not one-shot prediction.

The paper formalizes RL for VLAs exactly in this multimodal form, with iterative trajectory rollout in the environment Anonymous ers [2025].

SimpleVLA-RL: Core Mechanism



Core recipe

For each task input:

1. sample multiple trajectories from the current VLA policy
2. execute them in the simulator
3. assign a simple outcome reward: success = 1, failure = 0
4. update the policy with group-relative policy optimization

Why this matters

…hod avoids hand-crafted dense reward engineering and uses only task completion as supervision.

Algorithm: Online RL Post-Training for a VLA

Algorithm 3: Lecture reconstruction of the SimpleVLA-RL post-training loop

Input: Pretrained VLA policy π_θ, task batch $\mathcal{D}$, simulator Env
Output: Post-trained policy

for each task input in $\mathcal{D}$ **do**
    sample G trajectories by stochastic rollout from π_θ
    execute trajectories in simulation
    compute binary outcome reward for each trajectory
    keep groups with mixed success / failure outcomes
    estimate group-relative advantages
    update policy with GRPO-style clipped objective
end

Interpretation

This algorithm is simple by design: no dense reward shaping, no value function, and no need to recollect a large human demonstration dataset.

Three Exploration Enhancements

1. Dynamic sampling

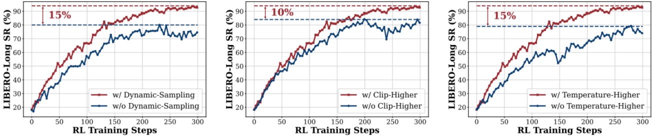
Ignore batches where all sampled trajectories either succeed or fail. This prevents zero-advantage collapse in group-relative RL.

2. Higher clipping range

Relax the PPO/GRPO upper clip boundary to allow stronger policy updates and more exploration.

3. Higher rollout temperature

Increase sampling temperature to generate more diverse trajectories during rollout.



(a) Dynamic Sampling **(b)** Clip Higher **(c)** Higher Rollout Temperature

The paper reports that these three modifications each improve LIBERO-Long success rate, with gains on the order of roughly 10–15% in their ablations Anonymous and Others [2025].

Benchmarks and Datasets

LIBERO

Language-guided manipulation benchmark with multiple suites:

- Spatial
- Object
- Goal
- Long
- LIBERO-90

The paper uses it to test long-horizon and generalization behavior.

RoboTwin 1.0 / 2.0

Simulation benchmark for dual-arm manipulation. RoboTwin 2.0 increases task diversity, domain randomization, and embodiment variety.

Why these datasets matter

- LIBERO tests language-conditioned manipulation transfer
- RoboTwin stresses longer horizons and bimanual coordination
- together they reveal whether RL improves more than one task family

Backbone used in the paper

SimpleVLA-RL is applied to **OpenVLA-OFT**, an autoregressive VLA variant based on OpenVLA with action chunking and parallel decoding.

The paper describes LIBERO, RoboTwin 1.0, and RoboTwin 2.0 explicitly, and states that its RL method is applied on top of OpenVLA-OFT Anonymous and Others [2025].

Main Results: RL Beats SFT

LIBERO

OpenVLA-OFT average:

91.0

After SimpleVLA-RL:

99.1

with gains across Spatial, Object, Goal, and Long suites.

RoboTwin 1.0

OpenVLA-OFT average:

39.8

After SimpleVLA-RL:

70.4

RoboTwin 2.0

OpenVLA-OFT average:

38.3

After SimpleVLA-RL:

68.8

Task Name	Steps	Horizon	Horizon Group
Short Horizon Tasks (112-130 steps)			
lift_pot	112	Short	Average: 121 steps Count: 4 tasks
beat_block_hammer	113	Short	
pick_dual_bottles	127	Short	
place_phone_stand	130	Short	
Medium Horizon Tasks (151-223 steps)			
move_can_pot	151	Medium	Average: 176 steps Count: 4 tasks
place_a2b_left	155	Medium	
place_empty_cup	174	Medium	
handover_mic	223	Medium	
Long Horizon Tasks (283-313 steps)			
handover_block	283	Long	Average: 298 steps Count: 2 tasks
stack_bowls_two	313	Long	
Extra Long Horizon Tasks (466-637 steps)			
blocks_rank_rgb	466	Extra-Long	Average: 552 steps Count: 2 tasks
put_bottles_dustbin	637	Extra-Long	
Overall Statistics	**Total: 12 tasks, Average: 256 steps**		

Data Efficiency and Generalization

One-trajectory setting

On LIBERO-Long, the paper reports:

$$17.3 \rightarrow 91.7$$

when moving from one-trajectory SFT to RL post-training.

Generalization

The paper claims stronger:

- spatial generalization
- object generalization
- goal generalization

than SFT-only baselines.

Interpretation

This is the central empirical message:

RL is not only optimization

RL is also capability amplification

The policy becomes more robust under sparse-data and out-of-distribution settings.

The one-trajectory jump and the spatial/object/goal generalization claims are highlighted in the paper overview and analysis sections Anonymous and Others [2025].

A Surprising Effect: “Pushcut”

Observation

During RL training, the policy sometimes discovers a previously unseen behavior pattern, called **pushcut** in the paper.

Why this matters

This is stronger than imitation. The policy is not merely copying demonstrations; it may discover a new strategy that still solves the task.

The paper explicitly names and illustrates this phenomenon as a novel behavior discovered during RL training Anonymous and Others [2025].

Paper Reading: SimpleVLA-RL

SimpleVLA-RL: Scaling VLA Training via Reinforcement Learning

What to notice while reading

- Why SFT scaling is not enough
- How RL is adapted from LLM-style post-training to robot policies
- Why binary outcome rewards can be enough
- Why exploration engineering matters
- Which gains come from RL, and which come from architecture

Key message for this course

Modern VLAs are becoming **post-trained policies**. The new frontier is not only better backbones, but better **interaction-driven improvement**.

Conclusions

Modern embodied AI systems are no longer static perception models; they are adaptive decision-making systems.

Perception $\longrightarrow$ Action $\longrightarrow$ Interaction $\longrightarrow$ Learning

Three structural shifts

1. training moves from single-task data to large multimodal mixtures
2. policy quality improves through interaction, not only supervision
3. performance depends on training pipelines as much as architecture

Where the field is moving (2026)

- robot foundation models
- simulation-driven training
- RL post-training
- autonomous capability improvement

Central lesson: modern AI systems learn not only from data, but from interaction with the world.

Bibliography I

Anonymous and Others. Simplevla-rl: Scaling vla training via reinforcement learning. **arXiv preprint arXiv:2509.09674**, 2025. URL https://arxiv.org/abs/2509.09674.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, et al. π_0: A vision-language-action flow model for general robot control. **arXiv preprint arXiv:2410.24164**, 2024. URL https://arxiv.org/abs/2410.24164.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Ryan Julian, Dmitry Kalashnikov, Sergey Levine, Yao Lu, Carolina Parada, Kanishka Rao, Pierre Sermanet, Fei Xia, and Ted Xu. Rt-1: Robotics transformer for real-world control at scale. **arXiv preprint arXiv:2212.06817**, 2022. URL https://arxiv.org/abs/2212.06817.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In **Conference on Robot Learning (CoRL)**, 2023. URL https://arxiv.org/abs/2307.15818.

Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Russ Tedrake. Diffusion policy: Visuomotor policy learning via action diffusion. **The International Journal of Robotics Research**, 42(14):1385–1407, 2023. URL https://arxiv.org/abs/2303.04137.

Bibliography II

Embodiment Collaboration, Abby O'Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, Albert Tung, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anchit Gupta, Andrew Wang, Andrey Kolobov, Anikait Singh, Animesh Garg, Aniruddha Kembhavi, Annie Xie, Anthony Brohan, Antonin Raffin, Archit Sharma, Arefeh Yavary, Arhan Jain, Ashwin Balakrishna, Ayzaan Wahid, Ben Burgess-Limerick, Beomjoon Kim, Bernhard Schölkopf, Blake Wulfe, Brian Ichter, Cewu Lu, Charles Xu, Charlotte Le, Chelsea Finn, Chen Wang, Chenfeng Xu, Cheng Chi, Chenguang Huang, Christine Chan, Christopher Agia, Chuer Pan, Chuyuan Fu, Coline Devin, Danfei Xu, Daniel Morton, Danny Driess, Daphne Chen, Deepak Pathak, Dhruv Shah, Dieter Büchler, Dinesh Jayaraman, Dmitry Kalashnikov, Dorsa Sadigh, Edward Johns, Ethan Foster, Fangchen Liu, Federico Ceola, Fei Xia, Feiyu Zhao, Felipe Vieira Frujeri, Freek Stulp, Gaoyue Zhou, Gaurav S. Sukhatme, Gautam Salhotra, Ge Yan, Gilbert Feng, Giulio Schiavi, Glen Berseth, Gregory Kahn, Guangwen Yang, Guanzhi Wang, Hao Su, Hao-Shu Fang, Haochen Shi, Henghui Bao, Heni Ben Amor, Henrik I Christensen, Hiroki Furuta, Homanga Bharadhwaj, Homer Walke, Hongjie Fang, Huy Ha, Igor Mordatch, Ilija Radosavovic, Isabel Leal, Jacky Liang, Jad Abou-Chakra, Jaehyung Kim, Jaimyn Drake, Jan Peters, Jan Schneider, Jasmine Hsu, Jay Vakil, Jeannette Bohg, Jeffrey Bingham, Jeffrey Wu, Jensen Gao, Jiaheng Hu, Jiajun Wu, Jialin Wu, Jiankai Sun, Jianlan Luo, Jiayuan Gu, Jie Tan, Jihoon Oh, Jimmy Wu, Jingpei Lu, Jingyun Yang, Jitendra Malik, João Silvério, Joey Hejna, Jonathan Booher, Jonathan Tompson, Jonathan Yang, Jordi Salvador, Joseph J. Lim, Junhyek Han, Kaiyuan Wang, Kanishka Rao, Karl Pertsch, Karol Hausman, Keegan Go, Keerthana Gopalakrishnan, Ken Goldberg, Kendra Byrne, Kenneth Oslund, Kento Kawaharazuka, Kevin Black, Kevin Lin, Kevin Zhang, Kiana Ehsani, Kiran Lekkala, Kirsty Ellis, Krishan Rana, Krishnan Srinivasan, Kuan Fang, Kunal Pratap Singh, Kuo-Hao Zeng, Kyle Hatch, Kyle Hsu, Laurent Itti, Lawrence Yunliang Chen, Lerrel Pinto, Li Fei-Fei, Liam Tan, Linxi "Jim" Fan, Lionel Ott, Lisa Lee, Luca Weihs, Magnum Chen, Marion Lepert, Marius Memmel, Masayoshi Tomizuka, Masha Itkina, Mateo Guaman Castro, Max Spero, Maximilian Du, Michael Ahn, Michael C. Yip, Mingtong Zhang, Mingyu Ding, Minho Heo, Mohan Kumar Srirama, Mohit Sharma, Moo Jin Kim, Muhammad Zubair Irshad, Naoaki Kanazawa, Nicklas Hansen, Nicolas Heess, Nikhil J Joshi, Niko Suenderhauf, Ning Liu, Norman Di Palo, Nur Muhammad Mahi Shafiullah, Oier Mees, Oliver Kroemer, Osbert Bastani, Pannag R Sanketi, Patrick "Tree" Miller, Patrick Yin, Paul Wohlhart, Peng Xu, Peter David Fagan, Peter Mitrano, Pierre Sermanet, Pieter Abbeel, Priya Sundaresan, Qiuyu Chen, Quan Vuong, Rafael Rafailov, Ran Tian, Ria Doshi, Roberto Martín-Martín, Rohan Baijal, Rosario Scalise, Rose Hendrix, Roy Lin, Runjia Qian, Ruohan Zhang, Russell Mendonca, Rutav Shah, Ryan Hoque, Ryan Julian,

Bibliography III

Samuel Bustamante, Sean Kirmani, Sergey Levine, Shan Lin, Sherry Moore, Shikhar Bahl, Shivin Dass, Shubham Sonawani, Shubham Tulsiani, Shuran Song, Sichun Xu, Siddhant Haldar, Siddharth Karamcheti, Simeon Adebola, Simon Guist, Soroush Nasiriany, Stefan Schaal, Stefan Welker, Stephen Tian, Subramanian Ramamoorthy, Sudeep Dasari, Suneel Belkhale, Sungjae Park, Suraj Nair, Suvir Mirchandani, Takayuki Osa, Tanmay Gupta, Tatsuya Harada, Tatsuya Matsushima, Ted Xiao, Thomas Kollar, Tianhe Yu, Tianli Ding, Todor Davchev, Tony Z. Zhao, Travis Armstrong, Trevor Darrell, Trinity Chung, Vidhi Jain, Vikash Kumar, Vincent Vanhoucke, Vitor Guizilini, Wei Zhan, Wenxuan Zhou, Wolfram Burgard, Xi Chen, Xiangyu Chen, Xiaolong Wang, Xinghao Zhu, Xinyang Geng, Xiyuan Liu, Xu Liangwei, Xuanlin Li, Yansong Pang, Yao Lu, Yecheng Jason Ma, Yejin Kim, Yevgen Chebotar, Yifan Zhou, Yifeng Zhu, Yilin Wu, Ying Xu, Yixuan Wang, Yonatan Bisk, Yongqiang Dou, Yoonyoung Cho, Youngwoon Lee, Yuchen Cui, Yue Cao, Yueh-Hua Wu, Yujin Tang, Yuke Zhu, Yunchu Zhang, Yunfan Jiang, Yunshuang Li, Yunzhu Li, Yusuke Iwasawa, Yutaka Matsuo, Zehan Ma, Zhuo Xu, Zichen Jeff Cui, Zichen Zhang, Zipeng Fu, and Zipeng Lin. Open x-embodiment: Robotic learning datasets and rt-x models, 2025. URL https://arxiv.org/abs/2310.08864.

Octo Model Collaboration et al. Octo: An open-source generalist robot policy. **arXiv preprint arXiv:2405.12213**, 2024. URL https://arxiv.org/abs/2405.12213.

Open X-Embodiment Collaboration. Open x-embodiment: Robotic learning datasets and rt-x models. **arXiv preprint arXiv:2310.08864**, 2023. URL https://arxiv.org/abs/2310.08864.

Physical Intelligence. $\pi_{0.5}$: A vision-language-action model with open-world generalization. **arXiv preprint arXiv:2504.16054**, 2025.

Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Oier Mees, Suraj Huang, et al. OpenVLA: An open-source vision-language-action model. **arXiv preprint arXiv:2406.09246**, 2024a. URL https://arxiv.org/abs/2406.09246.

Moo Jin Kim et al. Openvla: An open-source vision-language-action model. **arXiv preprint arXiv:2406.09246**, 2024b.